Systems Architecture

Characters / Strings

Text Characters

- One Byte per character
- Standard codes: ASCII
  American Standard Code for Information Interchange
- Defined in the 60s using a teletype model
- Special characters:

NUL	No char	*HT*	Horizontal Tab
BEL	Bell (Beep)	*LF*	Line Feed
BS	Backspace	*FF*	Form (Page) Feed
DEL	Delete	*CR*	Carriage Return
ESC	Escape	*SP*	Space

ASCII Character Codes

	MSB							
LSB	0	1	2	3	4	5	6	7
0	NUL	DLE	SP	0	@	P	‘	p
1	SOH	DC1	!	1	A	Q	a	q
2	STX	DC2	"	2	B	R	b	r
3	ETX	DC3	#	3	C	S	c	s
4	EOT	DC4	$	4	D	T	d	t
5	ENQ	NAK	%	5	E	U	e	u
6	ACK	SYN	&	6	F	V	f	v
7	BEL	ETB	’	7	G	W	g	w
8	BS	CAN	(	8	H	X	h	x
9	HT	EM	)	9	I	Y	i	y
A	LF	SUB	*	:	J	Z	j	z
B	VT	ESC	+	;	K	[	k	{
C	FF	FS	,	<	L	\	l	|
D	CR	GS	-	=	M	]	m	}
E	SO	RS	.	>	N	^	n	~
F	SI	US	/	?	0	_	o	DEL

0x20 = Space

0x30 = ’0’

0x41 = ’A’

0x61 = ’a’

Special Characters in C

`\t`	HT	Horizontal Tab	`\n`	LF (NL)	Line Feed (New Line)
`\b`	BS	Back Space	`\r`	CR	Carriage Return
`\a`	BEL	Audible Alarm	`\f`	FF	Form Feed
`\0`	NUL	no character	`\\`	`\`	Backslash
`\'`	`'`	Single Quote	`\"`	`"`	Double Quote

`\`*ddd* Character code *ddd*

123 Decimal

0173 Octal

0x7B Hex

Strings

- A *string* is a sequence of characters
- Three types of string:
  - ⇒ Fixed Length String
    String is of a fixed length (*n* characters)
    String is truncated if too long
    String is padded out with spaces if too short
  - ⇒ Terminated String
    Special character to mark end of string
    Normally a NUL (0x00) is used
    Used by C based languages (C, C++, Java, C#, . . .)
  - ⇒ Counted String
    Starts with a length value (character count)
    Used by Fortran based languages (Basic, Pascal, ADA, . . .)

Unicode: Unified Character Codes

Pate vs Pâté

- Superset of ASCII
- International Standard (ISO 10646)
- Two parts to a character:
  1) Character set indicator (1 or 3 bytes)
  2) Character from character set (1 bytes)
- Unicode16 — Two bytes per character
  All major character sets included
  Still under development
- Unicode32 — Four bytes per character
  Extended character set only

Unicode in C (wide characters)

- Include *wchar* header file
  `#include <wchar.h>`
- Literals:
  `L'â'` — Single wide character
  `L"Pâté"` — Wide String
- *wide* functions:
  `putwchar`, `getwchar`, `iswalpha`, ...
- Additional *wide* string functions:
  `strlen` — Length of string in bytes
  `wcslen` — Length of wide char string in chars
- International strings:
  `#include <libintl.h>`
  `gettext("Hello")` — *Hello* in current language

Unicode Text Format (Files)

UTF-32 Four bytes per character
Full (extended) character sets

UTF-16 Two bytes per character
Standard character sets only (not extended)

BOM Byte order mark
First character of file is 0xFEFF
Used to determine sex of the file:

EF BB BF	UTF-8
FE FF	UTF-16 Big Endian
FF FE	UTF-16 Little Endian
00 00 FE FF	UTF-32 Big Endian
FF FE 00 00	UTF-32 Little Endian

Unicode Files: UTF8

- Most common Unicode format
  Endian neutral
  Backward compatible with ASCII files
- Non-ASCII characters are encoded:

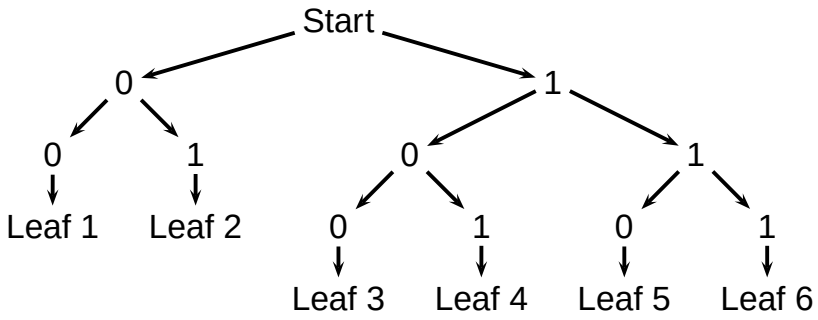
1st of 1	0xxx xxxx	0000 007F
1st of 2	110x xxxx	0000 07FF
1st of 3	1110 xxxx	0000 FFFF
1st of 4	1111 0xxx	001F FFFF
1st of 5	1111 10xx	03FF FFFF
1st of 6	1111 110x	7FFF FFFF
next of n	10xx xxxx	

FEFF ⇒ EF BB BF

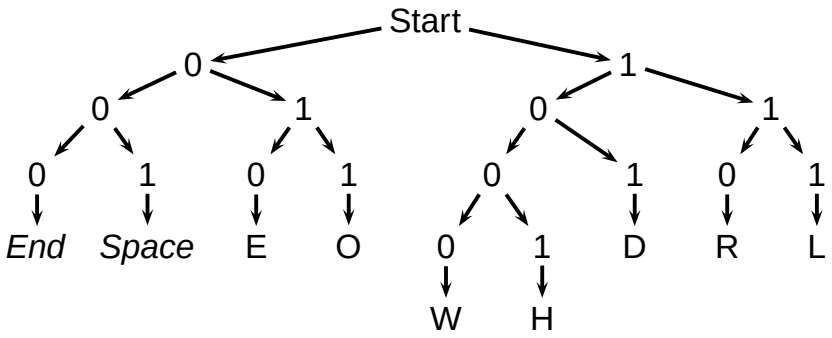
1111 1110 1111 1111 ⇒ 1110 1111 1011 1011 1011 1111

Huffman Codes

- Bit Stream
- Variable width codes
- Frequent codes given short values
- Arithmetic not possible



Huffman Example



1001	010	111	111	011	001	1000	011	110	111	101	000
H	E	L	L	O	*Space*	W	O	R	L	D	*End*

Uncompressed: 12 Bytes
Compressed: 38 bits (4.75 Bytes)
Compression Ratio: $(12 \div 100) \times 5 = 60\%$